

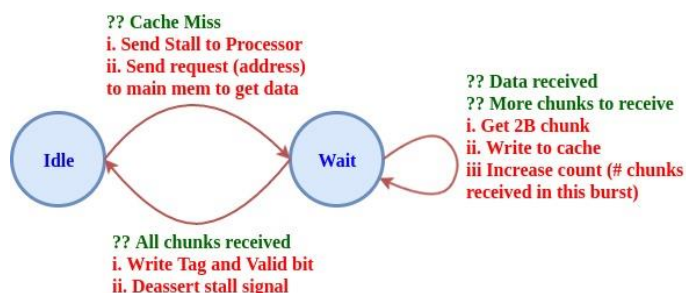
Department of Electrical and Computer Engineering
University of Wisconsin – Madison
ECE 552 Introduction to Computer Architecture
Homework 8 (Due 11/10)

For the question below (Verilog based), paste your entire Verilog code in a single text/doc file and submit the text/doc file on canvas. **Note that this Verilog question needs to follow the Verilog Rules specified in the 552 project/Verilog rules document that can be found on Canvas.**

1. In this homework, you are required to design (in Verilog) the controller for handling cache misses.

The following information may be required (some of it is specifically required for the homework, while the rest of it establishes context for the project; refer to the Project Phase 3 handout for more details):

- a. The data and address ports of the cache and memory are 16 bits wide each. The cache capacity is 2KB (i.e., 2048B), memory is 64KB (i.e., 65536B), and block size is 16 bytes.
- b. On both read and write misses, you need to bring in the correct block from memory to the cache.
- c. A single data transfer between cache and memory is of an entire cache block and is thus 16 bytes but the data transfer granularity (i.e., width of data ports in cache/memory and wires between them) is only 2-byte words. So you will need to sequentially grab 8 chunks of data from memory and insert them word-by-word into the cache block.
- d. Each 2-byte word of data requires 4 cycles to be read from memory; i.e., memory access latency is 4 cycles.
- e. The cache is write-through, so there is no data to be written back from the cache to memory upon eviction.
- f. The cache controller stalls the processor on a miss, and only after the entire cache block is brought into the cache is the stall signal deasserted.



The FSM for the controller is shown above. The blue refers to the state, the green refers to the condition for changing state, and the red refers to actions performed on state transitions.

The module interface for the controller is suggested below (you can make changes if necessary):

```
module cache_fill_fsm(
    input clk,
    input rst,
    input miss_detected, // active high when tag match logic detects a miss
    input [15:0] miss_address, // address that missed the cache
    output fsm_busy, // asserted while FSM is busy handling the miss (can be used as
pipeline stall signal)
    output write_data_array, // write enable to cache data array to signal when filling
with memory_data
    output write_tag_array, // write enable to cache tag array to signal when all words
are filled in to data array
    output [15:0] memory_address, // address to read from memory
    input [15:0] memory_data, // data returned by memory (after delay)
    input memory_data_valid // active high indicates valid data returning on memory bus
);
```

Solution:

```
module cache_fill_fsm(
    input clk,
    input rst,
    input miss_detected, // active high when tag match logic detects a miss
    input [15:0] miss_address, // address that missed the cache
    output reg fsm_busy, // asserted while FSM is busy handling the miss (can be used as pipeline
stall signal)
    output reg write_data_array, // write enable to cache data array to signal when filling with
memory_data
    output reg write_tag_array, // write enable to cache tag array to signal when all words are
filled in to data array
    output [15:0] memory_address, // address to read from memory
    input [15:0] memory_data, // data returned by memory (after delay)
    input memory_data_valid // active high indicates valid data returning on memory bus
);
```

```
reg cnt_en, cnt_clear, store_miss_address; // Signals to control counter and address storage
reg [3:0] cnt_out; // 4-bit counter output for chunk counting
reg [15:0] arithmetic_mem_address; // Register to hold incremented memory address
```

```
typedef enum logic {IDLE, WAIT} state_t; // Define FSM states as IDLE and WAIT
state_t current_state, next_state; // Declare registers to hold current and next state
```

```

// Stores either the miss address or incremented address based on the state of
store_miss_address
pldff #(.WIDTH(16)) addr_dut(
    .q(memory_address), // Output memory address
    .d(store_miss_address ? miss_address : arithmetic_mem_address), // Select between miss
address and incremented address
    .wen(cnt_en), // Enable writing to memory address
    .clk(clk),
    .rst(rst)
);

// Address Incrementer
// Adds 2 to the current memory address, used to increment the address in each cycle
addsub_16bit addr_increment_dut(
    .a(memory_address), // Input current memory address
    .b(16'h2), // Increment value (2 for each chunk)
    .c_in(1'b0), // No carry-in
    .sum(arithmetic_mem_address), // Output incremented address
    .ovfl() // Overflow (not used here)
);

// Counter for chunk counting
// Counts up to a specified number of chunks (here 8) and resets as needed
pldff #(.WIDTH(4)) cnt_dut(
    .q(cnt_out), // Counter output
    .d(cnt_out + 1), // Increment counter by 1
    .wen(cnt_en), // Enable counter when needed
    .clk(clk),
    .rst(rst | cnt_clear) // Reset counter on clear signal or reset
);

// FSM State Register
// Holds the current state of the FSM
dff state_reg(
    .q(current_state), // Output current state
    .d(next_state), // Next state to transition to
    .wen(1'b1), // Always enabled to update the state
    .clk(clk),
    .rst(rst) // Reset the state on reset
);

// FSM Logic
always @(*) begin
// Default values for outputs and control signals
    fsm_busy = 1'b0;

```

```

write_data_array = 1'b0;
write_tag_array = 1'b0;
store_miss_address = 1'b0;
cnt_en = 1'b0;
cnt_clear = 1'b0;
next_state = current_state; // By default, stay in the current state

case (current_state)

    IDLE: begin
        if (miss_detected) begin // If a cache miss is detected
            fsm_busy = 1'b1; // Set FSM busy signal
            store_miss_address = 1'b1; // Store the miss address
            cnt_en = 1'b1; // Enable the counter
            cnt_clear = 1'b1; // Clear the counter to start fresh
            next_state = WAIT; // Transition to WAIT state
        end
    end

    WAIT: begin
        fsm_busy = 1'b1; // FSM remains busy
        if (memory_data_valid) begin // If memory data is valid
            write_data_array = 1'b1; // Write to data array
            cnt_en = 1'b1; // Enable counter to increment
            if (cnt_out == 4'h8) begin // If all chunks are filled (counter reaches 8)
                write_tag_array = 1'b1; // Write to tag array to complete the fill
                next_state = IDLE; // Transition back to IDLE state
            end
        end
    end

endcase
end
endmodule

```